

Aussagenlogik - Teil 2

Formale Grundlagen

Finn Wellershaus

HAW Hamburg, Department Informatik

Version vom 25. Juni 2024



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Äquivalenz

Definition (Äquivalenz)

Zwei Formeln F und G heißen **äquivalent** genau dann, wenn jede Belegung beiden Formeln den gleichen Wahrheitswert zuweist, wenn also $\mathcal{A}(F) = \mathcal{A}(G)$ für jede Belegung $\mathcal{A}$ gilt.

Notation: $F \equiv G$.

Anmerkung

- 1 Alternativ: Zwei Formeln sind genau dann äquivalent, wenn sie dieselben Modelle besitzen, also $\mathcal{A}(F) = 1$ gdw. $\mathcal{A}(G) = 1$ gilt.
- 2 Äquivalente Formeln haben denselben Wahrheitswertverlauf!

Äquivalenz: Beispiel 1

Beweis von $(A \Leftrightarrow B) \equiv ((A \wedge B) \vee (\neg A \wedge \neg B))$ mit Wahrheitstafel:

A	B	$A \Leftrightarrow B$	$A \wedge B$	$\neg A \wedge \neg B$	$(A \wedge B) \vee (\neg A \wedge \neg B)$
0	0	1	0	1	1
0	1	0	0	0	0
1	0	0	0	0	0
1	1	1	1	0	1

In der dritten und letzten Spalte sieht man, dass $A \Leftrightarrow B$ und $(A \wedge B) \vee (\neg A \wedge \neg B)$ den gleichen Wahrheitswerteverlauf haben. Damit sind die beiden Formeln äquivalent.

Äquivalenz: Beispiel 2

Widerlegung von $A \wedge B \equiv A \vee B$ durch Angabe eines Gegenbeispiels:

$A \wedge B$ und $A \vee B$ sind nicht äquivalent, da z.B. $\mathcal{A}$ mit $\mathcal{A}(A) = 0$ und $\mathcal{A}(B) = 1$ zwar ein Modell von $A \vee B$ nicht aber eines von $A \wedge B$ ist.

Damit haben die beiden Formeln nicht die gleichen Modelle und sind damit nicht äquivalent.

Wichtige Anmerkung

Ebenso widerlegt man Folgerbarkeitsbeziehungen $F \models G$ durch Angabe eines Gegenbeispiels.

Also: durch Angabe einer Belegung $\mathcal{A}$, die Modell für F ist, aber nicht für G .

Wichtige Äquivalenzen

Kommutativität:	$(F \wedge G)$	$\equiv$	$(G \wedge F)$
	$(F \vee G)$	$\equiv$	$(G \vee F)$
Assoziativität:	$(F \wedge (G \wedge H))$	$\equiv$	$((F \wedge G) \wedge H)$
	$(F \vee (G \vee H))$	$\equiv$	$((F \vee G) \vee H)$
Distributivität:	$(F \wedge (G \vee H))$	$\equiv$	$((F \wedge G) \vee (F \wedge H))$
	$(F \vee (G \wedge H))$	$\equiv$	$((F \vee G) \wedge (F \vee H))$
Doppelnegation:	$\neg\neg F$	$\equiv$	F
de Morgans Regeln:	$\neg(F \wedge G)$	$\equiv$	$(\neg F \vee \neg G)$
	$\neg(F \vee G)$	$\equiv$	$(\neg F \wedge \neg G)$
Elimination von $\Leftrightarrow$:	$(F \Leftrightarrow G)$	$\equiv$	$(F \Rightarrow G) \wedge (G \Rightarrow F)$
	$(F \Leftrightarrow G)$	$\equiv$	$(F \wedge G) \vee (\neg F \wedge \neg G)$
Elimination von $\Rightarrow$:	$(F \Rightarrow G)$	$\equiv$	$(\neg F \vee G)$

Weitere wichtige Äquivalenzen

Absorption:	$(F \wedge (F \vee G))$	$\equiv$	F
	$(F \vee (F \wedge G))$	$\equiv$	F
Idempotenz:	$(F \wedge F)$	$\equiv$	F
	$(F \vee F)$	$\equiv$	F
Tautologieregeln	$(F \wedge \top)$	$\equiv$	F
	$(F \vee \top)$	$\equiv$	$\top$
Kontradiktionsregeln:	$(F \wedge \perp)$	$\equiv$	$\perp$
	$(F \vee \perp)$	$\equiv$	F
Komplement:	$(F \wedge \neg F)$	$\equiv$	$\perp$
	$(F \vee \neg F)$	$\equiv$	$\top$

Anmerkung

Wichtige Anmerkung

- 1 Auf der letzten Folien waren $\perp$ und $\top$ Konstanten.
Man müsste sie streng formal als neue syntaktische Konstrukte einführen.
Sie sind dann atomare Formeln, die immer zu 0 (bei $\perp$) bzw. immer zu 1 (bei $\top$) ausgewertet werden.
- 2 Alle obigen Äquivalenzen kann man z.B. mit Wahrheitstabeln schnell beweisen.
- 3 Bislang haben wir noch nicht bewiesen, dass wir auch in Teilformeln Äquivalenzumformungen vornehmen dürfen.
Dies leistet das **Ersetzungstheorem** (s.u.).

Klammerersparnisregeln

Aufgrund der Äquivalenzen können wir uns auf folgende Regeln zur Klammerersparnis einigen:

- 1 Die äußersten Klammern entfallen: $A \vee B$ statt $(A \vee B)$
- 2 Bei mehrfacher Konjunktion oder Disjunktion entfällt die mehrfache Klammerung: $((A \vee B) \vee C) \equiv (A \vee (B \vee C)) \equiv A \vee B \vee C$
- 3 Weiterhin **nicht** erlaubt sind $A \Rightarrow B \Rightarrow C$ oder $A \vee B \wedge C$

Bemerkung

In einigen Büchern findet man auch die Regel, dass $\neg$ am stärksten bindet, dann $\wedge$ und $\vee$ und als dritte $\Rightarrow$ und $\Leftrightarrow$.

Damit wäre dann z.B. auch $A \wedge B \Rightarrow C$ möglich.

Wir wollen dies i.A. aber nicht benutzen.

Eine Ausnahme sind Hornformeln in Implikationsschreibweise, zu denen wir später noch kommen.

Fragen

Wenn $F \equiv G$ und $\models G$ gilt, dann gilt auch $\models F$

- 1 Ja!
- 2 Nein!
- 3 Weiß ich nicht ...

Fragen

Wenn $F \models$ und $G \models$ gilt, dann gilt $F \equiv G$

- 1 Ja!
- 2 Nein!
- 3 Weiß ich nicht ...

Einige Sätze

Satz

- ① Wenn $F \equiv G$ und $\models G$ gilt, dann gilt auch $\models F$
- ② Wenn $F \equiv G$ und $G \models$ gilt, dann gilt auch $F \models$
- ③ Wenn $\models F$ und $\models G$ gilt, dann gilt $F \equiv G$
- ④ Wenn $F \models$ und $G \models$ gilt, dann gilt $F \equiv G$

Beweis.

1. und 4. nachfolgend. 2. und 3. in den Präsenzaufgaben.



Einige Sätze: Beweis zu (1)

Satz

Wenn $F \equiv G$ und $\models G$ gilt, dann gilt auch $\models F$.

Beweis.

Sie $\mathcal{A}$ eine zu F und G passende Belegung.

Da G eine Tautologie ist, gilt $\mathcal{A}(G) = 1$.

Da ferner $F \equiv G$ gilt, gilt $\mathcal{A}(F) = \mathcal{A}(G)$.

Also auch $\mathcal{A}(F) = 1$.

Damit ist jede Belegung Modell für F und folglich F eine Tautologie. □

Einige Sätze: Beweis zu (4)

Satz

Wenn $F \models$ und $G \models$ gilt, dann gilt $F \equiv G$

Beweis.

Wir zeigen, dass $\mathcal{A}(F) = \mathcal{A}(G)$ für jede Belegung $\mathcal{A}$ gilt.

Sei dazu $\mathcal{A}$ eine zu F und G passende Belegung.

Da sowohl F als auch G Kontradiktionen sind, gilt stets $\mathcal{A}(F) = \mathcal{A}(G) = 0$
womit bereits alles gezeigt ist. □

Sätze mit Folgerbarkeit

Satz

$F \equiv G$ genau dann, wenn $F \models G$ und $G \models F$

Beweis.

- Sei $F \equiv G$ und sei $\mathcal{A}$ ein Modell für F .
Wegen $F \equiv G$ ist $\mathcal{A}$ auch ein Modell für G .
Damit gilt $F \models G$. Analog gilt auch $G \models F$.
- Gelte nun andersherum $F \models G$ und $G \models F$ und sei $\mathcal{A}$ ein Modell für F .
Wegen $F \models G$ gilt dann auch $\mathcal{A}(G) = 1$.
Ist $\mathcal{A}'$ ein Modell für G , dann ist $\mathcal{A}'$ wegen $G \models F$ auch ein Modell für F .
Damit haben F und G genau dieselben Modelle und folglich gilt $F \equiv G$.



Drei wichtige Sätze

Satz

- ❶ $F \equiv G$ genau dann, wenn $\models (F \Leftrightarrow G)$
- ❷ $F \models G$ genau dann, wenn $\models (F \Rightarrow G)$
- ❸ $F \models G$ genau dann, wenn $(F \wedge \neg G) \models$

Beweis.

1. als Übung.

Zu 2: Gelte $F \models G$ und sei $\mathcal{A}$ eine zu F und G passende Belegung.

Ist $\mathcal{A}(F) = 0$, so ist $\mathcal{A}(F \Rightarrow G) = 1$ aufgrund der Definition der Semantik von $\Rightarrow$.

Ist $\mathcal{A}(F) = 1$, so ist wegen $F \models G$ auch $\mathcal{A}(G) = 1$ und damit wieder $\mathcal{A}(F \Rightarrow G) = 1$.

Folglich gilt $\models F \Rightarrow G$.

Sei umgekehrt $\models (F \Rightarrow G)$ und sei $\mathcal{A}$ ein Modell für F .

Wegen der Definition der Semantik von $\Rightarrow$ folgt aus $\mathcal{A}(F \Rightarrow G) = 1$ (es gilt ja

$\models (F \Rightarrow G)$) sofort $\mathcal{A}(G) = 1$

Damit ist jedes Modell von F auch Modell von G und wir sind fertig.

3. folgt aus 2. sofort aus der Äquivalenz $F \Rightarrow G \equiv \neg F \vee G$ und da die Negation einer Tautologie eine Kontradiktion ist (und umgekehrt) und da $\neg(\neg F \vee G) \equiv F \wedge \neg G$. $\square$

Verallgemeinerung

Die Aussagen

- ❶ $F \models G$ genau dann, wenn $\models (F \Rightarrow G)$
- ❷ $F \models G$ genau dann, wenn $(F \wedge \neg G) \models$

können verallgemeinert werden.

Sei M eine beliebige Formelmenge, dann gilt:

- ❶ $M \cup \{F\} \models G$ genau dann, wenn $M \models F \Rightarrow G$
- ❷ $M \models G$ genau dann, wenn $M \cup \{\neg G\}$ unerfüllbar ist.

Dabei ist eine Formelmenge unerfüllbar, wenn es keine Belegung gibt, die alle Formeln der Menge wahr macht.

Alle Sätze im Überblick

Satz

- 1 Wenn $F \equiv G$ und $\models G$ gilt, dann gilt auch $\models F$
- 2 Wenn $F \equiv G$ und $G \models$ gilt, dann gilt auch $F \models$
- 3 Wenn $\models F$ und $\models G$ gilt, dann gilt $F \equiv G$
- 4 Wenn $F \models$ und $G \models$ gilt, dann gilt $F \equiv G$
- 5 $F \equiv G$ genau dann, wenn $F \models G$ und $G \models F$
- 6 Wenn $F_1 \equiv F_2$ und $G_1 \equiv G_2$ gilt, dann gilt $F_1 \models G_1$ genau dann, wenn $F_2 \models G_2$ gilt.
- 7 $F \equiv G$ genau dann, wenn $\models (F \Leftrightarrow G)$
- 8 $F \models G$ genau dann, wenn $\models (F \Rightarrow G)$
- 9 $F \models G$ genau dann, wenn $F \wedge \neg G \models$
- 10 $M \cup \{F\} \models G$ genau dann, wenn $M \models (F \Rightarrow G)$
- 11 $M \models G$ genau dann, wenn $M \cup \{\neg G\}$ unerfüllbar ist.

Normalformen

- Wir wollen jetzt die Äquivalenzen nutzen, um Teilformeln zu ersetzen.
- Meistens machen wir dies, damit die Formel „irgendwie einfacher“ wird.
- Manchmal wollen wir die Formel aber in eine standardisierte Darstellung bringen.
- Eine solche **Normalform** ist nicht unbedingt einfacher oder kürzer.
- Es gibt verschiedene Normalformen – je nach Zweck.

Normalformen (2)

Eine Normalform hat verschiedene Vorteile:

- Einfach strukturiert (daher gut für Beweis, Algorithmen, ...)
- Eigenschaften lassen sich bisweilen leichter ablesen/ermitteln.
- Notwendig: Aus jeder Formel sollte eine Normalform durch (Äquivalenz)-Umformungen konstruierbar sein.
- Wünschenswert: Zu jeder Formel gibt es genau eine äquivalente Normalform.

Wir untersuchen zwei Normalformen: KNF und DNF. Diese sind stets konstruierbar, aber nicht eindeutig.

Ersetzungen

Unser Ziel ist es zunächst Teilformeln ersetzen zu dürfen, also aus

$$A \wedge (B \Rightarrow C)$$

z.B.

$$A \wedge (\neg B \vee C)$$

zu machen.

Die Rechtfertigung dafür wird die Äquivalenz $B \Rightarrow C \equiv \neg B \vee C$ sein.

Dass wir dies tatsächlich in Teilformeln so ersetzen dürfen, sagt uns das
Ersetzbarkeitstheorem.

Ersetzbarkeitstheorem

Satz (Ersetzbarkeitstheorem)

Seien F und G äquivalente Formeln und sei H eine Formel mit (mindestens) einem Vorkommen der Formel F als Teilformel.

Die Formel H' entsteht aus H , indem ein Vorkommen von F (in H) durch G ersetzt wird.

Dann sind H und H' äquivalent.

Also: Ersetzt man also eine Teilformel F einer Formel H durch eine äquivalente Formel, so ist die entstehende Formel H' zur ursprünglichen H äquivalent.

Der Satz ist die Rechtfertigung für die Schreibweise

$$A \wedge (B \Rightarrow C) \equiv A \wedge (\neg B \vee C).$$

Der Beweis erfolgt nun mittels struktureller Induktion...

Beweis

Induktionsanfang. Sei H eine atomare Formel und gehe H' durch Ersetzen von F durch G aus H hervor.

Dann muss $H = F$ und $H' = G$ gelten.

Aus $F \equiv G$ folgt dann sofort $H = F \equiv G = H'$. Also $H \equiv H'$.

Induktionsannahme. Wir nehmen an, dass H_1 und H_2 Formeln sind, für die gilt:

Für jede Formel H'_1 bzw. H'_2 , die durch Ersetzung von F durch G aus H_1 bzw. H_2 hervorgegangen ist, gilt $H_1 \equiv H'_1$ bzw. $H_2 \equiv H'_2$.

Induktionsschritt. Ist $F = H$, so verfahren wir wie im Induktionsanfang und sind fertig.

Nachfolgend genügt es also den Fall zu betrachten, dass F eine **echte** Teilformel von H ist.

- Fall $H = \neg H_1$.

Dann ist F eine Teilformel von H_1 .

Es gibt H'_1 , das aus H_1 durch Ersetzen von F durch G entsteht. Also:
 $H' = \neg H'_1$.

Nach Induktionsannahme gilt $H_1 \equiv H'_1$.

Sei nun $\mathcal{A}$ eine beliebige Belegung.

Dann ist $\mathcal{A}(H'_1) = \mathcal{A}(H_1)$ wegen $H_1 \equiv H'_1$.

Wegen der Definition der Semantik von $\neg$ ist dann $\mathcal{A}(\neg H'_1) = \mathcal{A}(\neg H_1)$.

Also auch $\mathcal{A}(H) = \mathcal{A}(H')$ und damit $H \equiv H'$.

- ..

Induktionsschritt (Fortsetzung).

- ..

- Fall $H = H_1 \vee H_2$.

Angenommen F ist eine Teilformel von H_1 (der andere Fall ist analog). Dann gibt es wieder ein H'_1 , das aus H_1 durch Ersetzen von F durch G entsteht. Also: $H' = H'_1 \vee H_2$.

Nach Induktionsannahme gilt $H_1 \equiv H'_1$.

Sei nun $\mathcal{A}$ eine beliebige Belegung.

Dann ist $\mathcal{A}(H'_1) = \mathcal{A}(H_1)$ wegen $H_1 \equiv H'_1$.

Ähnlich wie oben folgt (wegen der Definition der Semantik von $\vee$):

$$\begin{aligned}\mathcal{A}(H') &= \mathcal{A}(H'_1 \vee H_2) \\ &= f_{\vee}(\mathcal{A}(H'_1), \mathcal{A}(H_2)) \\ &= f_{\vee}(\mathcal{A}(H_1), \mathcal{A}(H_2)) \\ &= \mathcal{A}(H_1 \vee H_2) \\ &= \mathcal{A}(H)\end{aligned}$$

Also: $H' \equiv H$.

- Die anderen Fälle gehen ganz analog, was den Beweis abschließt.

Äquivalenzumformungen

Ergebnis

Wir dürfen nun dank des Ersetzbarkeitstheorems Teilformeln durch andere Formeln ersetzen, sofern diese zu der gewählten Teilformel äquivalent sind.

$$\neg A \Rightarrow \neg \neg B \equiv \neg A \Rightarrow B \equiv \neg \neg A \vee B \equiv A \vee B$$

Normalformen - Motivation

Wir wollen nun eine **Normalform** für aussagenlogische Formeln einführen, d.h. eine Form ...

- ... in die wir jede aussagenlogische Formel durch Äquivalenzumformungen bringen können und
- ... die eine praktische Form hat (z.B. für Berechnungen).

Dazu erst ein paar Begriffe ...

Literale und Klauseln

Definition

- 1 Ein **Literal** ist eine atomare Formel oder eine negierte atomare Formel.
- 2 Ein **positives Literal** ist eine atomare Formel, ein **negatives Literal** eine negierte atomare Formel.
- 3 Zwei Literale heißen **komplementär**, wenn sie positives und negatives Literal der gleichen atomaren Formel sind. Bspw. ist A das komplementäre Literal zu $\neg A$ und umgekehrt.
- 4 Literale und Disjunktionen von Literalen werden als **Klauseln** bezeichnet.
- 5 Literale und Konjunktionen von Literalen werden als **duale Klauseln** bezeichnet.

Notation für Klauseln:

$$\bigvee_{i=1}^n L_i = L_1 \vee \dots \vee L_n$$

Normalformen - KNF und DNF

Definition

- ① Eine Formel F ist in **konjunktiver Normalform** (KNF), wenn sie eine Konjunktion von Klauseln ist.

Also eine Konjunktion von Disjunktionen von Literalen, z.B.

$$(\neg A \vee B \vee \neg C) \wedge B \wedge (\neg C \vee \neg B) \wedge (A \vee B \vee C)$$

- ② Eine Formel F ist in **disjunktiver Normalform** (DNF), wenn sie eine Disjunktion von dualen Klauseln ist.

Also eine Disjunktion von Konjunktionen von Literalen, z.B.

$$(A \wedge B) \vee (\neg A \wedge \neg C) \vee (B \wedge \neg A \wedge C)$$

Eigenschaften der KNF und DNF

Merkhilfe

KNF: $(\neg A \vee B \vee \neg C) \wedge B \wedge (\neg C \vee \neg B)$

DNF: $(A \wedge B) \vee (\neg A \wedge \neg C) \vee (B \wedge \neg A \wedge C)$

Satz

- 1 *Eine KNF ist gültig
gdw. alle ihre Klauseln gültig sind
gdw. in allen Klauseln mindestens ein Paar komplementäre Literale
vorkommt.*
- 2 *Eine DNF ist unerfüllbar
gdw. alle ihre dualen Klauseln unerfüllbar sind
gdw. in allen dualen Klauseln mindestens ein Paar komplementäre
Literale vorkommt.*
- 3 *Ein Erfüllbarkeitstest für DNFs ist effizient implementierbar (Laufzeit ist
in P).*
- 4 *Für KNFs gilt dies wahrscheinlich nicht! Denn dies Problem ist
NP-vollständig.*

KNF und DNF

Aus jeder Formel kann eine Normalform durch (Äquivalenz)-Umformungen konstruierbar sein.

Satz

Zu jeder Formel gibt es (mindestens) eine konjunktive Normalform und (mindestens) eine disjunktive Normalform.

Formaler: Zu jeder Formel F gibt eine Formel K in konjunktiver Normalform und eine Formel D in disjunktiver Normalform mit $F \equiv K \equiv D$.

Aus F können K und D effektiv konstruiert werden.

Notiz: Die Formulierung „effektiv konstruierbar“ meint, dass es einen Algorithmus gibt.

Beweis.

Der Beweis ist wieder mittels struktureller Induktion möglich. Siehe *Logik für Informatiker* von Schöning.

Wir verfahren hier anders und geben eine Konstruktion an...



Beispiel

$$\begin{aligned}(A \wedge (B \Rightarrow C)) &\Rightarrow D \\ \equiv (A \wedge (\neg B \vee C)) &\Rightarrow D \\ \equiv \neg(A \wedge (\neg B \vee C)) \vee D \\ \equiv (\neg A \vee \neg(\neg B \vee C)) \vee D \\ \equiv (\neg A \vee (\neg\neg B \wedge \neg C)) \vee D \\ \equiv (\neg A \vee (B \wedge \neg C)) \vee D \\ \equiv ((\neg A \vee B) \wedge (\neg A \vee \neg C)) \vee D \\ \equiv ((\neg A \vee B) \vee D) \wedge ((\neg A \vee \neg C) \vee D) \\ \equiv (\neg A \vee B \vee D) \wedge (\neg A \vee \neg C \vee D)\end{aligned}$$

Elimination von $\Rightarrow$

Elimination von $\Rightarrow$

de Morgan

de Morgan

Doppelte Negation

Distributivität

Distributivität

Klammern

KNF und DNF - Existenzbeweis

Wir konstruieren zu F die KNF K und die DNF D mittels Äquivalenzumformungen wie folgt:

- 1 Forme F so um, dass nur die Junktoren $\neg$, $\wedge$ und $\vee$ vorkommen.
- 2 Forme weiter so um, dass Negationen nur vor atomaren Formeln vorkommen.
- 3 Forme mittels der Distributivgesetze weiter so um, dass eine DNF bzw. KNF entsteht.

KNF und DNF - Existenzbeweis

Die Schritte im einzelnen:

Schritt 1. Forme F so um, dass nur die Junktoren $\neg$, $\wedge$ und $\vee$ vorkommen, indem die anderen Junktoren durch sie ausgedrückt werden.

Z.B. kann $F \Rightarrow G$ durch $\neg F \vee G$ ersetzt werden und $F \Leftrightarrow G$ durch $(\neg F \vee G) \wedge (\neg G \vee F)$.

Wir erhalten die Formel F_1 .

Schritt 2. Wir formen F_1 durch wiederholte Anwendung von 'doppelte Negation' ($\neg\neg F \equiv F$) und 'de Morgan' ($\neg(F \wedge G) \equiv \neg F \vee \neg G$ und $\neg(F \vee G) \equiv \neg F \wedge \neg G$) um.

Wir erhalten eine Formel F_2 , deren Negationen nur noch vor den Atomen vorkommen.

KNF und DNF - Existenzbeweis

Schritt 3a. KNF. Wir formen F_2 durch wiederholte Anwendung des Distributivgesetzes in eine KNF um:

$$(F \vee (G \wedge H)) \equiv ((F \vee G) \wedge (F \vee H))$$

$$((F \wedge G) \vee H) \equiv ((F \vee H) \wedge (G \vee H))$$

Schritt 3b. DNF. Wir formen F_2 durch wiederholte Anwendung des Distributivgesetzes in eine DNF um:

$$(F \wedge (G \vee H)) \equiv ((F \wedge G) \vee (F \wedge H))$$

$$((F \vee G) \wedge H) \equiv ((F \wedge H) \vee (G \wedge H))$$

KNF und DNF - Existenzbeweis

Man beachte folgendes:

- ➊ In jedem Schritt werden Äquivalenzumformungen vorgenommen.
Die entstehenden Formeln sind also zur ursprünglichen äquivalent (Ersetzbarkeitstheorem).
- ➋ Jeder Schritt terminiert:
 - ➊ Im ersten Schritt gibt es nur endlich viele $\Rightarrow$ und $\Leftrightarrow$ zu ersetzen.
 - ➋ Im zweiten Schritt 'rutschen' die Negationen stets eine Ebene tiefer, so dass nur endlich oft umgeformt werden kann.
 - ➌ Im dritten Schritt 'rutschen' die Disjunktionen (bei der DNF) bzw. die Konjunktionen (bei der KNF) eine Ebene höher, so dass nur endlich oft umgeformt werden kann.
- ➌ Am Ende entsteht nach Konstruktion eine DNF bzw. eine KNF.

Damit haben wir ein Verfahren, das **terminiert**, eine **äquivalente** Formel liefert, die zudem in DNF bzw. KNF ist. Damit ist das Verfahren korrekt.

Bilden Sie zu

$$F := (A \Rightarrow B) \wedge (\neg C \Rightarrow \neg B)$$

durch Äquivalenzumformungen **nach obigem Verfahren** (a) eine äquivalente Formel in disjunktiver Normalform und (b) eine in konjunktiver Normalform.

Wahrheitstafelmethode

Eine weitere Möglichkeit eine DNF und eine KNF zu einer Formel zu konstruieren, geht mit **Wahrheitstafeln**.

Anmerkung

Hier ist das Problem, dass die Wahrheitstafeln sehr groß werden können und das Verfahren daher ineffizient.

Wahrheitstafelmethode - DNF

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Idee für die DNF?

Wahrheitstafelmethode - DNF

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1 $\neg A \wedge B \wedge C$
1	0	0	1 $A \wedge \neg B \wedge \neg C$
1	0	1	0
1	1	0	0
1	1	1	1 $A \wedge B \wedge C$

$$(\neg A \wedge B \wedge C) \quad \vee$$

$$(A \wedge \neg B \wedge \neg C) \quad \vee$$

$$(A \wedge B \wedge C)$$

Wahrheitstafelmethode - KNF

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

KNF ... ?

Wahrheitstafelmethode - KNF

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$A \vee B \vee C$$

$$A \vee B \vee \neg C$$

$$A \vee \neg B \vee C$$

$$\neg A \vee B \vee \neg C$$

$$\neg A \vee \neg B \vee C$$

$$(A \vee B \vee C) \wedge$$

$$(A \vee B \vee \neg C) \wedge$$

$$(A \vee \neg B \vee C) \wedge$$

$$(\neg A \vee B \vee \neg C) \wedge$$

$$(\neg A \vee \neg B \vee C)$$

Wahrheitstafelmethode - DNF

Herstellung der DNF:

- ➊ Erstelle pro Zeile, die zu 1 ausgewertet wird, eine duale Klausel:
 - ➊ Wird ein Aussagesymbole A zu 1 ausgewertet, nehme A in die Klausel
 - ➋ Wird ein Aussagesymbole A zu 0 ausgewertet, nehme $\neg A$ in die Klausel
- ➋ Verknüpfe alle so gewonnenen dualen Klauseln mit $\vee$
- ➌ Gibt es keine Zeile, die zu 1 ausgewertet wird, nimm die Formel $A \wedge \neg A$

Wahrheitstafelmethode - KNF

Herstellung der KNF:

- ➊ Erstelle pro Zeile, die zu 0 ausgewertet wird, eine Klausel:
 - ➊ Wird ein Aussagesymbole A zu 1 ausgewertet, nehme $\neg A$ in die Klausel
 - ➋ Wird ein Aussagesymbole A zu 0 ausgewertet, nehme A in die Klausel
- ➋ Verknüpfe alle so gewonnenen Klauseln mit $\wedge$
- ➌ Gibt es keine Zeile, die zu 0 ausgewertet wird, nimm die Formel $A \vee \neg A$

Wahrheitstafelmethode

Korrektheit

- Bei der Wahrheitstafelmethode werden offensichtlich DNFs bzw. KNFs erstellt.
- Diese sind zudem äquivalent zur ursprünglichen Formel, wie man sich schnell überlegt.
- So erzeugt man bei der DNF mit den einzelnen dualen Klauseln gerade Formeln, die genau an der Stelle der Zeile zu 1 (sonst 0en) ausgewertet werden.
- Durch die $\vee$ -Verknüpfung hat man dann gerade eine zur ursprünglichen Formel äquivalente Formel.
- Bei der Erstellung der KNF hat zunächst die einzelne Klausel gerade an der Stelle der Zeile eine 0 (sonst 1en).
- Durch die $\wedge$ -Verknüpfung hat man dann gerade wieder eine zur ursprünglichen Formel äquivalente Formel.

Erfüllbarkeitsäquivalenz

Wenn wir nur Erfüllbarkeit testen wollen, benötigen wir nicht unbedingt Äquivalenzen.

Uns würde *Erfüllbarkeitsäquivalenz* reichen, d.h. Umformungen die den Status der Erfüllbarkeit nicht verändern.

Definition (Erfüllbarkeitsäquivalenz)

Zwei Formeln F und G sind **erfüllbarkeitsäquivalent**, wenn F genau dann erfüllbar ist, wenn G dies ist.

Wir notieren dies als $F \equiv_e G$.

Anmerkung

Zwei Formeln sind also dann erfüllbarkeitsäquivalent, wenn sie beide erfüllbar sind oder auch wenn sie beide unerfüllbar sind.

Wegen letzterem ist diese Eigenschaft für z.B. Resolution ausreichend!

Erfüllbarkeitsäquivalenz: KNF

Problem

Formt man eine DNF in KNF um, dann vergrößert sich die Formel exponentiell:

$$(x_1 \wedge y_1) \vee \dots \vee (x_n \wedge y_n) \quad \mapsto \quad (x_1 \vee \dots \vee x_n) \wedge \dots \wedge (y_1 \vee \dots \vee y_n)$$

Die Laufzeiten (z.B. für den Test auf Erfüllbarkeit) vergrößern sich dementsprechend auch exponentiell.

Abhilfe: Wir konstruieren aus einer beliebigen Formel eine **erfüllbarkeitsäquivalente** KNF.

Schritt 1: Wir treiben (wie zuvor) die Negationen bis an die Atome.

Die Formellänge kann sich dabei höchstens verdoppeln.

Schritt 2: Konjunktionen und Disjunktionen.

- Wenn die Formel die Form $(F \wedge G)$ hat:
Rekursiv erzeugen wir in poly-Zeit dazu die erfüllbarkeitsäquivalenten KNF F' und G' , d.h. $F' \equiv_e F$ und $G' \equiv_e G$.
Die gesuchte KNF ist dann $(F' \wedge G')$.
- Wenn die Formel die Form $(F \vee G)$ hat:

Trick: $(F \vee G) \equiv_e ((F \vee A) \wedge (G \vee \neg A))$ für ein frisches Atom A .

Für die KNF $F' = F_1 \wedge \dots \wedge F_k$ ist dann

$(F \vee A) \equiv (F_1 \vee A) \wedge \dots \wedge (F_k \vee A) =: F''$ wieder eine KNF.

Analog für G . Also erhalten wir (in poly-Zeit) die KNF:

$$(F \vee G) \equiv_e ((F' \vee A) \wedge (G' \vee \neg A)) \equiv_e (F'' \wedge G'')$$

Wir erhalten somit zu jeder Formel eine erfüllbarkeitsäquivalente KNF.

Diese wird sich nur polynomiell vergrößern.

Motivation

Erfüllbarkeit zu prüfen ist bei ...

- ...KNFs schwierig (genauer: *NP*-vollständig)
- ...aussagenlogischen Formeln allgemein schwierig (folgt aus obigem)
- ...DNFs einfach

Aber: Eine DNF herzustellen ist (wegen obigem) schwierig (bzw. dauert lange).

Anmerkung

Zusammengefasst: Erfüllbarkeit zu testen ist bereits für Aussagenlogik schwierig.

Somit sind auch Algorithmen für Folgerung etc. zeitaufwändig.

Motivation

Erfüllbarkeit ist *im allgemeinen* für Aussagenlogik ein komplexes Problem.

Konsequenz

Man sucht daher nach **Einschränkungen der Aussagenlogik**, so dass ..

- die noch zur Verfügung stehenden Formeln weiterhin **möglichst ausdrucksmächtig** sind,
- aber Erfüllbarkeit **schnell** zu überprüfen ist.

Hornformeln sind so eine Einschränkung.

Hornformeln

Definition (Hornklauseln und Hornformeln)

- ① Eine Klausel $K = (L_1 \vee \dots \vee L_n)$ ist genau dann eine **Hornklausel**, wenn K höchstens ein positives Literal enthält.
- ② Eine Formel in KNF, bei der jede Klausel eine Hornklausel ist, ist eine **Hornformel**.

Beispiel

- ① Hornklauseln: $A, \neg A \vee B, \neg A \vee B \vee \neg C, \neg A \vee \neg B$
- ② Keine Hornklausel: $A \vee B, \neg A \vee B \vee C$
- ③ Hornformel: $A \wedge (\neg A \vee B) \wedge (\neg A \vee \neg B \vee \neg C)$

Typen von Hornklauseln

Es gibt drei Typen von Hornklauseln. Für alle kann eine **Implikationsschreibweise** benutzt werden:

- ❶ K enthält negative Literale und genau ein positives Literal (Regeln):

$$\begin{aligned} K &= \neg A_1 \vee \neg A_2 \vee \dots \vee \neg A_i \vee A_k \\ &\equiv (\neg A_1 \vee \neg A_2 \vee \dots \vee \neg A_i) \vee A_k \\ &\equiv \neg(A_1 \wedge A_2 \wedge \dots \wedge A_i) \vee A_k \\ &\equiv (A_1 \wedge A_2 \wedge \dots \wedge A_i) \Rightarrow A_k \end{aligned}$$

- ❷ K enthält nur negative Literale (Beschränkungen):

$$\begin{aligned} K &= \neg A_1 \vee \neg A_2 \vee \dots \vee \neg A_i \equiv (\neg A_1 \vee \neg A_2 \vee \dots \vee \neg A_i) \vee \perp \\ &\equiv \neg(A_1 \wedge A_2 \wedge \dots \wedge A_i) \vee \perp \equiv (A_1 \wedge A_2 \wedge \dots \wedge A_i) \Rightarrow \perp \end{aligned}$$

- ❸ K enthält nur genau ein positives Literal (Fakten):

$$K = A_1 \equiv \neg \top \vee A_1 \equiv \top \Rightarrow A_1$$

Zu Hornformeln

Definition

- 1 Eine Hornklausel, in der kein Aussagensymbol mehrfach auftritt, heißt reduziert.
- 2 Eine Hornformel heißt reduziert genau dann, wenn alle Klauseln reduziert und verschieden sind.

Satz

Für jede (nicht allgemeingültige) Hornformel existiert eine äquivalente reduzierte Hornformel.

Satz

Es gibt keine zu $A \vee B$ äquivalente Hornformel.

Zu Hornformeln

- Hornformeln sind also in gewisser Weise recht eingeschränkt.
- Dennoch geht mit ihnen ziemlich viel ($\Rightarrow$ Logik-Programmierung).
- Daher lohnt es sich, sich Gedanken über eine effizienten Erfüllbarkeitstest zu machen.

Ideen?

$$(\neg A \vee C) \wedge A \wedge (\neg A \vee \neg C)$$

bzw.

$$(A \Rightarrow C) \wedge (\top \Rightarrow A) \wedge ((A \wedge C) \Rightarrow \perp)$$

Markierungsalgorithmus für Hornformeln

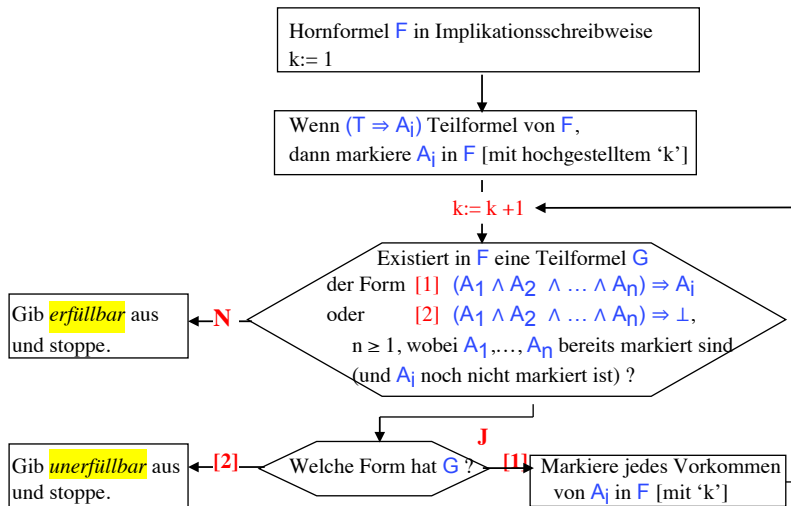
Grundidee: Markiere Aussagesymbole, die in jeder erfüllenden Belegung wahr sein müssen.

- ① Versehe jedes Vorkommen eines Aussagesymbols A_i in F mit einer Markierung, falls es in F eine Teilformel der Form $(\top \Rightarrow A_i)$ gibt.
- ② WHILE es gibt in F eine Teilformel G der Form
 - [1] $(A_1 \wedge A_2 \wedge \dots \wedge A_n) \Rightarrow A_i$ oder der Form
 - [2] $(A_1 \wedge A_2 \wedge \dots \wedge A_n) \Rightarrow \perp$,wobei $A_1, \dots, A_n, n \geq 1$ bereits markiert sind (und A_i noch nicht)
DO
 - IF G hat die Form [1]
 - THEN markiere jedes Vorkommen von A_i in F
 - ELSE gib “unerfüllbar” aus und stoppe.
- ③ Gib “erfüllbar” aus und stoppe.

Die gefundene erfüllende Belegung ist:

$$\mathcal{A}(A_i) = \begin{cases} 1, & \text{falls } A_i \text{ eine Markierung besitzt} \\ 0, & \text{sonst} \end{cases}$$

Markierungsalgorithmus (Ablaufdiagramm)



Beispiel

$$\begin{aligned} & (\neg A \vee \neg B \vee E) \wedge C \wedge A \wedge (\neg C \vee B) \wedge (\neg A \vee D) \\ & \wedge (\neg C \vee \neg D \vee A) \wedge \neg E \wedge \neg G \\ \equiv & ((A \wedge B) \Rightarrow E) \wedge (\top \Rightarrow C) \\ & \wedge (\top \Rightarrow A) \wedge (C \Rightarrow B) \\ & \wedge (A \Rightarrow D) \wedge ((C \wedge D) \Rightarrow A) \\ & \wedge (E \Rightarrow \perp) \wedge (G \Rightarrow \perp) \end{aligned}$$

Markierungen:

- 1. Runde: A und C
- 2. Runde: B (wegen $C \Rightarrow B$) und D (wegen $A \Rightarrow D$)
- 3. Runde: E (wegen $(A \wedge B) \Rightarrow E$)
- 4. Runde: **unerfüllbar** (wegen $(E \Rightarrow \perp)$)

Beispiel

$$\begin{aligned} & (\neg A \vee B) \wedge A \wedge (\neg B \vee \neg A \vee C) \wedge (\neg D \vee C) \\ \equiv & (A \Rightarrow B) \wedge (\top \Rightarrow A) \wedge ((A \wedge B) \Rightarrow C) \wedge (D \Rightarrow C) \end{aligned}$$

Markierungen:

- 1. Runde: A
- 2. Runde: B (wegen $(A \Rightarrow B)$)
- 3. Runde: C (wegen $((A \wedge B) \Rightarrow C)$)
- 4. Runde: Keine weiteren Markierungen.

Erfüllende Belegung $\mathcal{A}$ mit $\mathcal{A}(A) = \mathcal{A}(B) = \mathcal{A}(C) = 1$ und $\mathcal{A}(D) = 0$.

Korrektheit des Markierungsalgorithmus für Hornformeln

Satz

Der Markierungsalgorithmus ist korrekt, d.h.:

- *Wenn F eine unerfüllbare Hornformel ist, dann gibt der Algorithmus unerfüllbar aus.*
- *Wenn F erfüllbar ist, so gibt der Algorithmus eine erfüllende Belegung aus.*

Ergebnis

Der Markierungsalgorithmus ist sehr effizient.

Gemessen an der Anzahl der atomaren Formeln benötigt er Linearzeit.

Er arbeitet aber nur für Hornformeln, aber nicht jede Formel ist als Hornformel darstellbar.

PROLOG

Prolog erlaubt es, mit Hornformel zu programmieren.

Betrachten wir die Hornformel:

$$((b \wedge c \wedge d) \Rightarrow a) \wedge ((d \wedge e) \Rightarrow a) \wedge d \wedge e$$

Diese wird in Prolog folgendermaßen ausgedrückt:

`a :- b, c, d.`

`a :- d, e.`

`d .`

`e .`

An der Kommandozeile können wir jetzt abfragen, ob bspw. *a* gilt:

`?- a.`

`Yes`

Prolog beherrscht darüberhinaus noch die Darstellung von Prädikaten, wie wir sie erst für Prädikatenlogik kennenlernen werden

Motivation

Wir wollen nun die **Resolution** betrachten, ein **spezielles Ableitungsverfahren**, das mit KNFs arbeitet.

Die Resolution ist ein **Widerlegungsverfahren**, d.h. es ist möglich eine Formel auf Unerfüllbarkeit zu testen.

Auf die Idee zur Resolution kann man kommen, wenn man einige der bisherigen Regeln betrachtet ...

Motivation

- Disjunktiver Syllogismus 1: $\frac{\neg G, F \vee G}{F}$
- Disjunktiver Syllogismus 2: $\frac{\neg F, F \vee G}{G}$
- Modus Ponens: $\frac{F, F \Rightarrow G}{G} \dots \frac{F, \neg F \vee G}{G}$
- Modus Tollens: $\frac{\neg G, F \Rightarrow G}{\neg F} \dots \frac{\neg G, \neg F \vee G}{\neg F}$
- Hypothetischer Syllogismus: $\frac{F \Rightarrow G, G \Rightarrow H}{F \Rightarrow H} \dots \frac{\neg F \vee G, \neg G \vee H}{\neg F \vee H}$

Es werden Pärchen komplementäre Literale entfernt.
Dies kann man verallgemeinern um zu einem korrekten Ableitungsverfahren zu gelangen.

Mengendarstellung

Resolution arbeitet mit Formeln in KNF und benutzt eine **Mengendarstellung**.

- Ist $K = \bigvee_{i=1}^m L_i$ eine Klausel, dann ist $\mathbf{K} = \{L_1, \dots, L_m\}$ die Mengendarstellung von K .
- Ist $F = (\bigwedge_{i=1}^n (\bigvee_{j=1}^{m_i} L_{i,j}))$ eine Formel in KNF, dann ist $\mathbf{F} = \{\{L_{1,1}, \dots, L_{1,m_1}\}, \dots, \{L_{n,1}, \dots, L_{n,m_n}\}\}$ die Mengendarstellung von F .
- Die leere Klausel (Mengendarstellung zu $\perp$) wird mit $\square$ bezeichnet.
- Die Wahrheitswerteberechnung wird angepasst:
 - $\mathcal{A}(\mathbf{K}) = \max(\{\mathcal{A}(L) \mid L \in \mathbf{K}\})$
 - $\mathcal{A}(\mathbf{F}) = \min(\{\mathcal{A}(\mathbf{K}) \mid \mathbf{K} \in \mathbf{F}\})$
- $\mathcal{A}(\square) = \mathcal{A}(\emptyset) = \max(\emptyset) = 0$
- In dieser Darstellung ist sogar $\mathbf{F} = \emptyset$ (d.h. eine Formel, die aus 0 Klauseln besteht) interpretierbar:
Sie ist wegen $\min(\emptyset) = 1$ stets wahr.

Mengendarstellung - Beispiel

$$F = (\neg A \vee B) \wedge E \wedge (\neg G \vee \neg C)$$

Dann ist

- $K_1 = \neg A \vee B$ und $\mathbf{K}_1 = \{\neg A, B\}$
- $K_2 = E$ und $\mathbf{K}_2 = \{E\}$
- $K_3 = \neg G \vee \neg C$ und $\mathbf{K}_3 = \{\neg G, \neg C\}$
- $\mathbf{F} = \{\{\neg A, B\}, \{E\}, \{\neg G, \neg C\}\}$

Anmerkung

In der Mengendarstellung sind die Junktoren nicht mehr explizit zu sehen.

Die Resolutionsregel

Definition (Resolvente)

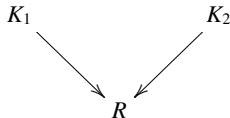
Seien K_1 und K_2 Klauseln (in Mengendarstellung) und L ein Literal mit $L \in K_1$ und $\bar{L} \in K_2$.

Dann heißt die (Literal-)Menge $R = (K_1 - \{L\}) \cup (K_2 - \{\bar{L}\})$ **Resolvente** von K_1 und K_2 (bzgl. L).

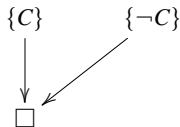
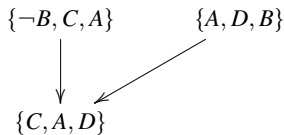
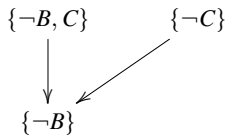
Dabei ist $\bar{L} := A$, falls $L = \neg A$ und $\bar{L} := \neg A$, falls $L = A$.

Im Falle $K_1 = \{L\}$ und $K_2 = \{\neg L\}$ ist $R = \emptyset$ und wird durch $\square$ symbolisiert.

- Resolventenbildung als Ableitung: $\{K_1, K_2\} \vdash_{res} R$
- Darstellung als Diagramm:



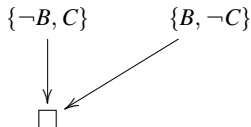
Beispiele



Beispiele

Wichtige Anmerkung

Das unten Stehende auf keinen Fall!



Wichtige Anmerkung

Wirklich nicht! Nein, nein, nein! Es wird immer nur genau ein Literal aus der einen Mengen und ein (komplementäres) Literal aus der anderen Menge entfernt!

Beispiele

Wichtige Anmerkung

Das unten Stehende auch nicht!

$$\{\neg B, B\}$$



Wichtige Anmerkung

Zur Resolventenbildung werden zwei Klauseln benötigt!

Resolventenmengen

Ähnlich wie bei den Ableitungen führen wir auch hier mehrschrittige Resolventenbildungen ein:

Definition

Sei F eine Formel in KNF dargestellt als Klauselmenge.

$$\begin{aligned} \text{Res}(F) &:= F \cup \{R \mid R \text{ ist Resolvente zweier Klauseln aus } F\} \\ \text{Res}^0(F) &:= F \\ \text{Res}^{n+1}(F) &:= \text{Res}(\text{Res}^n(F)) \\ \text{Res}^*(F) &:= \bigcup_{i \geq 0} \text{Res}^i(F) \end{aligned}$$

Zweck des Ganzen (zeigen wir später):

Satz (Resolutionssatz)

Eine Klauselmenge F ist unerfüllbar genau dann, wenn $\square \in \text{Res}^(F)$ gilt.*

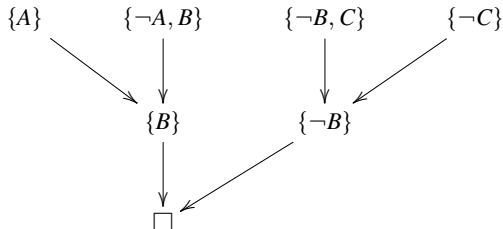
Beispiel

Wir wollen die folgende Formel auf Erfüllbarkeit testen:

$$(\neg A \vee B) \wedge (\neg B \vee C) \wedge A \wedge \neg C$$

Als Klauselmenge:

$$\{\{\neg A, B\}, \{\neg B, C\}, \{A\}, \{\neg C\}\}$$



- Wir zeigen nun den Resolutionssatz und
- leiten daraus einen Algorithmus zum Test auf Unerfüllbarkeit ab.

Lemma (Resolutionslemma)

Sei F eine Formeln in KNF als Klauselmeng und R eine Resolvente zweier Klauseln K_1 und K_2 in F . Dann sind F und $F \cup \{R\}$ äquivalent.

Lemma

Sei F eine Formel in KNF als Klauselmeng. Dann gilt

$$F \equiv Res^1(F) \equiv Res^2(F) \equiv \dots \equiv Res^n(F) \equiv \dots \equiv Res^*(F)$$

Satz (Resolutionssatz)

Eine Klauselmeng F ist unerfüllbar genau dann, wenn $\square \in Res^(F)$ gilt.*

Mengendarstellung - Wiederholung

Resolution arbeitet mit Formeln in KNF und benutzt eine **Mengendarstellung**

- Ist $K = \bigvee_{i=1}^m L_i$ eine Klausel, dann ist $\mathbf{K} = \{L_1, \dots, L_m\}$ die Mengendarstellung von K .
- Ist $F = (\bigwedge_{i=1}^n (\bigvee_{j=1}^{m_i} L_{i,j}))$ eine Formel in KNF, dann ist $\mathbf{F} = \{\{L_{1,1}, \dots, L_{1,m_1}\}, \dots, \{L_{n,1}, \dots, L_{n,m_n}\}\}$ die Mengendarstellung von F .
- Die leere Klausel (Mengendarstellung zu $\perp$) wird mit $\square$ bezeichnet.
- Die Wahrheitswerteberechnung wird angepasst:
 - $\mathcal{A}(\mathbf{K}) = \max(\{\mathcal{A}(L) \mid L \in \mathbf{K}\})$
 - $\mathcal{A}(\mathbf{F}) = \min(\{\mathcal{A}(\mathbf{K}) \mid \mathbf{K} \in \mathbf{F}\})$
 - $\mathcal{A}(\square) = 0$

Das Resolutionslemma

Satz (Resolutionslemma)

Sei F eine Formeln in KNF und R eine Resolvente zweier Klauseln K_1 und K_2 in F . Dann sind F und $F \cup \{R\}$ äquivalent.

Beweis.

Sei $\mathcal{A}$ eine Belegung. Gilt $\mathcal{A} \models F \cup \{R\}$, so gilt sofort auch $\mathcal{A} \models F$. (Warum?)

Gelte umgekehrt $\mathcal{A} \models F$, d.h. es ist $\mathcal{A} \models K$ für alle Klauseln $K \in F$.

Sei $R = (K_1 - \{L\}) \cup (K_2 - \{\bar{L}\})$ mit $K_1, K_2 \in F$ und $L \in K_1, \bar{L} \in K_2$.

Nun gibt es zwei Fälle:

- ① $\mathcal{A} \models L$. Dann folgt $\mathcal{A} \models K_2 - \{\bar{L}\}$ aus $\mathcal{A} \models K_2$ und $\mathcal{A} \models \bar{L}$.
- ② $\mathcal{A} \not\models L$. Dann folgt $\mathcal{A} \models K_1 - \{L\}$ aus $\mathcal{A} \models K_1$.

In beiden Fällen folgt $\mathcal{A} \models R$ und damit insgesamt $\mathcal{A} \models F \cup \{R\}$. □

Ein Lemma/Korollar

Lemma

Sei F eine Formel in KNF als Klauselmengen. Dann gilt

$$F \equiv \text{Res}^1(F) \equiv \text{Res}^2(F) \equiv \dots \equiv \text{Res}^n(F) \equiv \dots \equiv \text{Res}^*(F)$$

Beweis.

Folgt sofort aus dem Resolutionslemma von eben ...



Der Resolutionssatz

Satz (Resolutionssatz)

Eine Klauselmenge F ist unerfüllbar genau dann, wenn $\square \in Res^(F)$ gilt.*

Beweis (1)

$\Leftarrow$: Angenommen $\square \in Res^*(F)$. Wir wollen zeigen, dass F unerfüllbar ist. Zunächst kann $\square$ nur durch die Resolution zweier Klauseln $K_1 = \{L\}$ und $K_2 = \{\bar{L}\}$ entstanden sein.

Wegen der Definition von $Res^*(F)$ muss es wegen $\square \in Res^*(F)$ ferner ein n geben mit $\square \in Res^n(F)$.

Damit gilt auch $K_1, K_2 \in Res^n(F)$.

Da keine Belegung sowohl K_1 als auch K_2 erfüllen kann, ist $Res^n(F)$ unerfüllbar.

Nach dem vorherigen Lemma gilt $F \equiv Res^n(F)$. Also ist auch F unerfüllbar. Dies zeigt die **Korrektheit** der Resolution.

Beweis (Fortsetzung)

$\Rightarrow$: Sei nun F als unerfüllbar angenommen. Wir wollen $\Box \in \text{Res}^*(F)$ zeigen. Dies gelingt mittels Induktion über die Anzahl n der in F vorkommenden atomaren Formeln.

Induktionsanfang. ist $n = 0$, so ist $F = \{\Box\}$ und wegen $F \subseteq \text{Res}^*(F)$ ist

$\Box \in F \subseteq \text{Res}^*(F)$.

Induktionsannahme. Angenommen, für jede Klauselmeng G , die nur n atomare Formeln enthält und die unerfüllbar ist, gilt $\Box \in \text{Res}^*(G)$.

Induktionsschritt. Sei F eine unerfüllbare Formelmeng, die die atomaren Formeln $A_1, \dots, A_{n+1}$ enthält. Zunächst konstruieren wir zwei Formeln F_0 und F_1 wie folgt:

- Idee: Für F_0 belegen wir A_{n+1} mit 0 und vereinfachen. F_0 entsteht aus F , indem wir jedes Vorkommen von A_{n+1} in einer Klausel streichen und bei Vorkommen von $\neg A_{n+1}$ die ganze Klausel streichen.
- Idee: Für F_1 belegen wir A_{n+1} mit 1 und vereinfachen.
 F_1 entsteht aus F , indem wir jedes Vorkommen von $\neg A_{n+1}$ in einer Klausel streichen und bei Vorkommen von A_{n+1} die ganze Klausel streichen.

Auf F_0 und F_1 ist nun die Induktionsannahme anwendbar, **wenn** wir denn zeigen können, dass sie unerfüllbar sind!

BdW: Angenommen F_0 hätte doch eine erfüllende Belegung $\mathcal{A}$.

Dann ist aber $\mathcal{A}'$ mit $\mathcal{A}'(B) = \mathcal{A}(B)$ für die $B \in \{A_1, \dots, A_n\}$ und $\mathcal{A}'(A_{n+1}) = 0$ eine erfüllende Belegung für F im Widerspruch zur Unerfüllbarkeit von F .

(Analog für F_1 .)

Beweis (Fortsetzung)

Es gilt also $\square \in \text{Res}^*(F_0)$ und $\square \in \text{Res}^*(F_1)$ nach Induktionsannahme.

D.h., es gibt für F_0 Klauseln $K_1, \dots, K_m$ mit

- $K_m = \square$ und für $i = 1, \dots, m$ gilt
- $K_i \in F_0$ oder K_i ist Resolvente von K_a, K_b , $a, b < i$

Ebenso gibt es für F_1 Klauseln $K'_1, \dots, K'_l$.

Wir stellen nun durch Wiedereinführung von A_{n+1} in die Klauseln $K_1, \dots, K_m$ dort, wo es gestrichen wurde, die ursprünglichen Klauseln wieder her.

❶ Aber: Zur Herstellung von F_0 wurden die Klauseln mit $\neg A_{n+1}$ gestrichen. Diese treten also bei den $K_1, \dots, K_m$ nicht auf!

❷ Es kann also nur das positive Literal A_i wieder eingeführt werden.

❸ Gilt hier bereits $\square \in \text{Res}^*(F)$, so sind wir bereits fertig!

Dies tritt auf, wenn in keiner der Klauseln $K_1, \dots, K_m$ ein A_{n+1} auftritt.

❹ Anderfalls erhalten wir eine Folge von Klauseln, die $\{A_{n+1}\} \in \text{Res}^*(F)$ bezeugt.

Analog erhält man aus den $K'_1, \dots, K'_l$ durch Wiedereinführen von $\neg A_{n+1}$ dort, wo es gelöscht wurde, eine Folge, die $\{\neg A_{n+1}\} \in \text{Res}^*(F)$ bezeugt.

Insg. haben wir damit $\{A_{n+1}\} \in \text{Res}^*(F)$ und $\{\neg A_{n+1}\} \in \text{Res}^*(F)$.

Die Resolvente dieser beiden Klauseln bezeugt dann $\square \in \text{Res}^*(F)$.

Dies zeigt die **Vollständigkeit** der Resolution.

qed.

Anmerkung: Resolution bei unendlichen Klauselmengen

Im Beweis gehen wir von einer endlichen Klauselmenge F aus.

Der Beweis geht auch mit einer unendlichen Klauselmenge $\mathcal{M}$.

Endlichkeitssatz (auch: Kompaktheitstheorem)

Eine Menge $\mathcal{M}$ von Formeln ist genau dann unerfüllbar, wenn mindestens eine **endliche** Teilmenge von $\mathcal{M}$ unerfüllbar ist.

Verfahren: Wir enumerieren alle endliche Teilmenge von $\mathcal{M}$ und testen, ob sie unerfüllbar sind. Bei der ersten unerfüllbaren Teilmenge wird abgebrochen.

- Korrektheit: Wird für eine endliche Teilmenge die leere Klausel erzeugt, dann ist die Teilmenge unerfüllbar und damit auch $\mathcal{M}$ selbst.
- Vollständigkeit: Ist umgekehrt $\mathcal{M}$ unerfüllbar, so wird irgendwann die richtige Teilmenge getestet und $\mathcal{M}$ so als unerfüllbar nachgewiesen.
- Sollte $\mathcal{M}$ dagegen erfüllbar sein, dann wird die leere Klausel nie generiert und die Enumeration endet nicht.

(Dass der Algorithmus dann nicht terminieren wird, ist von der obigen Formulierung erlaubt.)

Anmerkung: Resolution bei unendlichen Klauselmengen

Herausforderung: Systematisches Generieren aller endlichen Teilmengen von $\mathcal{M}$.

Betrachte die Formelmenge $\mathcal{L}_{AL}^{l,m}$, die eine Formel $F \in \mathcal{L}_{AL}$ enthält, wenn gilt:

- ① Die Formel hat eine Länge von höchstens l .
- ② Die Formel basiert nur auf den Atomen $A_1, \dots, A_m$.

Es gilt:

- Zu jeder Formel $F \in \mathcal{L}_{AL}$ existiert ein (l, m) , so dass $F \in \mathcal{L}_{AL}^{l,m}$.
- Zu jeder **endlichen** Formelmenge $M \subseteq \mathcal{L}_{AL}$ existiert ein (l, m) , so dass $M \subseteq \mathcal{L}_{AL}^{l,m}$.
- Zu jedem Paar (l, m) ist die Anzahl der konstruierbaren Formeln endlich, denn durch die zweite Eigenschaft ist unser Alphabet endlich geworden und wir betrachten nur Worte bis zu einer festen Länge.
- Damit ist dann zu festem (l, m) die Menge $\mathcal{P}(\mathcal{L}_{AL}^{l,m})$ aller Mengen von solchen Formeln ebenfalls endlich.

Wir iterieren nun nach Cantor systematisch alle Paare (l, m) und generieren alle Teilmengen $M \in \mathcal{P}(\mathcal{L}_{AL}^{l,m})$ in einer beliebigen, aber festen Reihenfolge.

Sollte diese Formelmenge M auch eine Teilmenge von $\mathcal{M}$ sein, so hat sie ihre Position gefunden und wird per Resolution überprüft; andernfalls wird sie übergangen.

Es entsteht eine surjektive Abbildung, d.h. wir erzeugen jede Teilmenge von $\mathcal{M}$.

Resolutionsalgorithmus

Ein Algorithmus zur Resolutionsbildung:

Algorithm 1 Resolutionsalgorithmus

```
1: Input: Eine Formel  $F$  in KNF
2: Bilde zu  $F$  eine Klauselmenge (auch  $F$  genannt)
3: repeat
4:    $G := F$ 
5:    $F := \text{Res}(F)$ 
6: until  $(\square \in F)$  or  $(F = G)$ 
7: if  $\square \in F$  then
8:   return  $F$  ist unerfüllbar
9: else
10:  return  $F$  ist erfüllbar
11: end if
```

Für Genießer: Dies ist eine **Fixpunktiteration**.

Termination

Der Resolutionsalgorithmus terminiert aufgrund des folgenden Satzes:

Satz

Für jede endliche Klauselmengen F gibt es ein $k \geq 0$ mit

$$Res^k(F) = Res^{k+1}(F) = \dots = Res^*(F)$$

Beweis.

Enthalte F die n atomaren Formeln $A_1, \dots, A_n$.

Da jede dieser n Formeln in einer Klausel positiv vorkommen kann, negativ vorkommen kann, positiv und negativ vorkommen kann oder gar nicht vorkommen kann, gibt es vier Möglichkeiten wie sie in einer Klausel auftreten kann.

Also existieren maximal 4^n viele Klauseln, die aus $A_1, \dots, A_n$ aufgebaut sein können.

Mehr können auch in $Res^*(F)$ nicht vorkommen.

Da in einem Schritt von $Res^i(F)$ zu $Res^{i+1}(F)$ mindestens eine Klausel hinzukommen muss, muss $k \leq 4^n$ gelten. □

Zum Algorithmus

- Man muss nicht alle Resolventen bilden, wenn man $\Box \in Res^*(F)$ zeigen will!
- Der Resolutionsalgorithmus ist in einigen Fällen sehr schnell.
- In anderen müssen (exponentiell) viele Klauseln erzeugt werden.
- I.A. ist der Algorithmus also bzgl. der Laufzeit mit dem Aufstellen der Wahrheitstabellen vergleichbar.
- Dies ist auch wenig überraschend: *SAT* ist *NP*-vollständig!

Dennoch gibt es nun Verfahren, deren Ziel es ist, die Resolution zu verbessern.

Diese Verfahren sind meist wieder korrekt und vollständig, schränken aber die Möglichkeiten Resolventen zu bilden ein - und werden dadurch **meist** schneller und übersichtlicher.

I.A. benötigen sie aber wie oben exponentielle Laufzeit.

Verfeinerungen

Definition (P-Resolution)

Eine Klausel heißt **positiv**, wenn sie nur positive Literale enthält.
Bei der **P-Resolution** darf nur dann eine Resolvente aus den Klauseln K_1 und K_2 gebildet werden, wenn eine der beiden Klauseln positiv ist.

Definition (N-Resolution)

Eine Klausel heißt **negativ**, wenn sie nur negative Literale enthält.
Bei der **N-Resolution** darf nur dann eine Resolvente aus den Klauseln K_1 und K_2 gebildet werden, wenn eine der beiden Klauseln negativ ist.

Satz

P- und N-Resolution ist vollständig (und weiterhin korrekt).

Satz

P-Resolution ist vollständig (und weiterhin korrekt).

Wir erhalten aus der KNF F die KNF $F_{A=0}$, indem wir jedes Vorkommen von A streichen und alle Klauseln mit $\neg A$ komplett streichen.
(Dies entspricht einer Auswertung von F mit $A \mapsto 0$.)

Beweis.

Sei F unerfüllbar. Induktion über die Anzahl n der Atome in F .

- Anfang $n = 0$: Dann ist $F = \{\square\}$ und wir sind fertig.
- Schritt $n > 0$: Sei A ein Atom in F .
- Die Spezialisierungen $F_{A=0}$ und $F_{A=1}$ sind beide unerfüllbar, denn andernfalls wäre F erfüllbar
- Nach IA ist $\square$ aus $F_{A=0}$ herleitbar.
- Fügen wir in dieser Herleitung die gestrichenen A 's wieder in die Klauseln ein, so erhalten wir eine Herleitung von $\{A\}$ aus F .
- Dies ist immer noch eine P-Resolution, da A positiv ist.
- Wir nutzen nun die Klausel $\{A\}$, um aus F durch P-Resolutionen alle $\neg A$ zu löschen und so die Klauseln in $F_{A=1}$ herzustellen.
- Nach IA ist aus $F_{A=1}$ die leere Klausel P-resolvierbar.

Also existiert für jede unerfüllbare Formel F ein P-Resolutionsbeweis. □

Verfeinerung

Definition (Einheitsresolution)

Bei der **Einheitsresolution** darf nur dann eine Resolvente aus den Klauseln K_1 und K_2 gebildet werden, wenn mindestens eine der beiden Klauseln aus nur genau einem Literal besteht.

Satz

Einheitsresolution ist vollständig für die Klasse der Hornformeln, aber nicht für beliebige KNF-Formeln. (Korrekt ist sie weiterhin.)

Prüfen Sie mittels Resolution, ob die folgenden Formeln erfüllbar sind:

❶ $F_1 := C \wedge (\neg C \vee A) \wedge (\neg C \vee B \vee \neg A) \wedge (\neg B \vee \neg A) \wedge (\neg D \vee B)$

❷ $F_2 := (\neg D \Rightarrow B) \wedge (A \vee B) \wedge (A \Rightarrow \neg B) \wedge \neg C \wedge ((A \wedge D) \Rightarrow C)$

Hinweis: Hier ist es sinnvoll, eine Resolutionsstrategie zu verwenden.

1 Äquivalenzen und Normalformen

- Folgerung vs. Äquivalenz
- Äquivalenzumformungen
- Normalformen: KNF und DNF
- Erfüllbarkeitsäquivalenz

2 Das AL-Fragment: Horn-Logik

- Hornformeln
- Markierungsalgorithmus
- PROLOG

3 Resolution in der Aussagenlogik

- KNF in Mengendarstellung
- Resolventen
- Resolutionsstrategien